

Double Deep Q-Network for Mountain Car

Offline-Initialized DDQN with Replay Buffer and Target Network

Vaideesh K. | CS 4122: Reinforcement Learning | Mahindra University | May 2026

Overview

This project implements Double Deep Q-Network (DDQN) from scratch to solve the Mountain Car environment from OpenAI Gymnasium. The environment is notoriously difficult due to sparse rewards: the agent only receives a reward when it reaches the goal, making random exploration almost useless.

Two variants were trained: a standard online DDQN, and one initialized with an offline dataset collected from human gameplay. The offline initialization gives the agent a head start by pre-filling the replay buffer with demonstrations before online interaction begins.

Course	CS 4122 - Reinforcement Learning
Environment	Mountain Car (OpenAI Gymnasium Classic Control)
Algorithm	Double DQN with Replay Buffer and Target Network
Tech Stack	Python, PyTorch, OpenAI Gymnasium
Key Files	training.py, testing.py, DQN_offline_false.pth, DQN_offline_true.pth
GitHub	github.com/Vaideesh-k/Double-DQN-Mountain-Car

Environment — Mountain Car

The Mountain Car environment has a car stuck in a valley. The goal is to reach the flag at the top of the right hill. The car's engine isn't strong enough to drive straight up — it has to build momentum by swinging back and forth.

Parameter	Value
Observation space	2 continuous values: position [-1.2, 0.6], velocity [-0.07, 0.07]
Action space	3 discrete: push left (0), no push (1), push right (2)
Reward	-1 per time step until goal reached (max 200 steps)
Termination	Car reaches position ≥ 0.5 , or 200 steps elapsed
Randomness	Initial position uniform in [-0.6, -0.4], initial velocity = 0

The sparse reward makes this hard to train. With random exploration, the agent almost never reaches the goal and gets -200 every episode with no gradient signal to improve. This is why offline data initialization helps significantly.

Algorithm — Double DQN

Standard DQN suffers from overestimation bias because it uses the same network to select and evaluate actions in the Bellman target. DDQN fixes this by decoupling the two:

- **Action selection:** Use the online (predict) network to pick the best next action.
- **Action evaluation:** Use the target network to get the Q-value of that action.

Bellman target for DDQN:

```
a* = argmax_a Q_online(s', a)
y = r + gamma * Q_target(s', a*) * (1 - done)
```

Key Components

Replay Buffer

Stores transitions (s, a, r, s', done) from environment interaction. At each training step, a random minibatch is sampled to break temporal correlations between consecutive transitions. Buffer size: 10,000.

Target Network

A copy of the online network whose weights are updated less frequently (every C steps). This stabilizes training by keeping the Bellman targets fixed for several updates rather than chasing a moving target.

Offline Data Initialization

When use_offline_data=True, the replay buffer is pre-filled with transitions from car_dataset.csv, collected via human gameplay using MountainCar_CollectOfflineData.py. For the first E episodes, no new data is added to the buffer — the agent trains purely on human demonstrations. After E episodes, online transitions are mixed in normally.

This gives the agent early exposure to successful trajectories (reaching the goal), providing useful gradient signal from the start instead of random -200 episodes.

Hyperparameters

Parameter	Value
Learning rate	1e-3

Discount factor (γ)	0.99
Replay buffer size	10,000
Batch size	64
Target network update	Every 10 episodes
Epsilon start	1.0
Epsilon end	0.05
Epsilon decay	Exponential over training
Offline warmup (E)	Tuned based on dataset quality
Network architecture	2 hidden layers, 64 neurons each, ReLU

Training

Without Offline Data (DQN_offline_false)

Standard online DDQN. The agent explores randomly at first and only starts learning once it occasionally reaches the goal by chance. Training is slow and unstable early on because of the sparse reward — the agent goes many episodes without any positive signal.

With Offline Data (DQN_offline_true)

The replay buffer is initialized with human gameplay data from `car_dataset.csv` (at least 30 episodes). The agent trains on this data for E episodes before mixing in online experience. Because the human data contains successful goal-reaching trajectories, the agent sees useful reward signal from episode 1 and converges significantly faster.

Reward curves for both variants are in `rewards_without_offline.png`.

Results

Both models successfully learned to solve Mountain Car. The offline-initialized variant showed faster early convergence due to pre-exposure to goal-reaching behavior in the replay buffer.

Final model sizes are under 100 KB each, using a lightweight 2-hidden-layer network. The final trained models are saved as `DQN_offline_false.pth` and `DQN_offline_true.pth`.

Testing is demonstrated in `testing.py`, which loads the final model and renders one full episode with animation.

Repository Structure

training.py	DDQN training loop — online and offline variants
testing.py	Load final model and render one episode
MountainCar_CollectOfflineData.py	Collect human gameplay data into car_dataset.csv
MountainCar_PlayGame.py	Play Mountain Car manually with keyboard
CartPole_DQN_training.py	Reference DQN implementation on CartPole
car_dataset.csv	Human gameplay dataset (30+ episodes)
DQN_offline_false.pth	Trained model — no offline data
DQN_offline_true.pth	Trained model — offline initialized
rewards_without_offline.png	Reward curves comparing both training runs
Report.pdf	Full implementation and analysis report

How to Run

Install dependencies:

```
pip install gymnasium torch numpy pygame
pip install swig && pip install gymnasium[classic-control]
```

Train both models:

```
python training.py
```

Test the trained model (renders animation):

```
python testing.py
```

Collect your own offline data:

```
python MountainCar_CollectOfflineData.py
```